

Malware Analysis
Analysis of WannaCry Ransomware Variant

Alejandro Panigada

CMP302: Advanced Ethical Hacking

2022/23

.

Note that Information contained in this document is for educational purposes.

.

Abstract

Among a collection of malware samples stored in a VMWare virtual environment, it was given the

task to choose one sample, conduct an analysis using whatever method was needed and report any findings.

Regardless of the method, sample behavior had to be observed, its inner architecture reconstructed, and actions performed on the victim machine collected and analyzed.

To achieve the goal of the sample a combination of static and dynamic analysis was conducted using several software packages, results collected in graphical form for later analysis, and finally an overview of both sample behavior and effects on the victim machine constructed alongside possible mitigations.

It was found that the chosen sample was a variant of the WannaCry Ransomware from 2017[1].

Unlike the original, this variant was found to have no worm component included, so no

propagation was attempted by the executed sample. The malware still has both the encryption and

C&C component present. Not having a worm component means that the severity of having one machine compromised by the sample is much lower and makes it easier to mitigate the threat.

Reverse engineering of the sample revealed several pieces of information about its inner architecture in some cases useful for better identifying the threat and having a more efficient mitigation.

.

1 CONTENT

2 Introduction 1

2.1 Background 1

2.2 Aim 2

3 Procedure 3

3.1 Overview of Procedure 3

3.2 General test 3

3.3 Planned Methodology 6

3.4 Dynamic Analysis 7

3.4.1 Debugging 7

3.4.2 Network Traffic Analysis 7

3.5 Static Analysis 8

3.5.1 PE File and Header Analysis 8

3.5.2 String Analysis 8

3.5.3 Disassembly 8

3.5.4 Analysis of artifacts produced by Execution 8

3.6 History of Procedure 9

3.6.1 Initial Inspection 9

3.6.2 Base Executable Analysis 12

3.6.3 Decrypting DLL Module 16

3.6.4 Encryption DLL Analysis 19

3.6.5 Network Traffic Analysis 22

3.6.6 Attempt at intercepting HTTPS request for Tor bundle download 22

4 Results 24

4.1 Base Executable Analysis 24

4.1.1 Activity Reconstruction 24

4.2 Encryption DLL Analysis 24

4.2.1 Activity Reconstruction 24

5 Discussion 25

5.1 General Discussion 25

5.2 Countermeasures 25

5.3 Future Work 25

.

References..... 26
Appendices..... 28
Appendix A..... 28
5.3.1 XIA Archive Content Identification..... 28
Appendix B..... 33
..... 33

.

2 INTRODUCTION

2.1 BACKGROUND

Malware is a threat to both individuals and corporate entities. Purposes for malware creation and propagation are many and their form varies based on the chosen target and motive.

Since whether it is an individual or a corporation, data stored on their machines is often an invaluable asset, ransomware is a dangerous threat. By using encryption, victims are denied access to their data leaving them only to decide whether paying the ransom is worth the risk.

Corporations that must keep their services online without any disturbance and often store important customer data, find themselves unable to restore both after the infection, often leading to important economic impact.

Individuals tend to use storage to save important personal data; if a backup is not available, it can be problematic if the malware reaches their machine.

In malware such as WannaCry, even if a ransom is paid, that does not lead to data being decrypted and restored. [2]

In short terms, both the creation and propagation of ransomware and malware in general is a common but severe threat, which creates both economic and personal problems for many parties and neutralization of it is necessary to prevent damage.

2.2 AIM

The document describes the analysis of a malware sample using several methods and software packages to both understand the underlying architecture of it as well any activity it performs on the victim machine. To do so evidence and information must be retrieved from the sample executable. This is achieved by performing:

- Reverse engineering of the malware sample with any appropriate method.
- Collection of any evidence regarding activity performed on the victim machine.
- Mapping of the sample executable using data obtained during reverse engineering.
- Analysis results and discuss them.

This is to be all performed inside a sandboxed virtual environment using VMWare Workstation software where the sample will not have access to any real valuable information and can be run with almost no restriction without compromising the user performing the analysis.

3 Procedure

3.1 Overview of Procedure

The provided virtual environment contained 6 samples. Each of them had no naming but was only identified by a number in the containing folder.

Sample 1 which is the folder named “1” inside the “Samples” directory was the one chosen.

The folder included only an executable PE file running under Windows. The operating system used by the virtual machine was Windows 10 with both Firewall and Windows Defender deactivated to avoid disruption of the analysis performed.

The virtual environment was provided with a snapshot used to restore the machine to a clean state whenever needed and had the network adapter set to host-only for security reasons.

Also, to prevent any unexpected and unintentional spreading of the malware to the host machine and beyond, the virtual network adapter connected to the virtual machine was disabled on the host.

Before establishing an exact procedure for analyzing the Sample, a general test was conducted.

It was needed to better understand the general effects on the victim machine caused by the execution. From information obtained in such a stage, a methodology would be established which would be adapted based on the observed behavior.

3.2 General test

Since actions taken for analyzing the sample would change depending on its behavior, it was first attempted to execute as naturally would without any tampering and observed the results on the affected virtual machine.

After execution, it was soon observed that the malware sample being analyzed was a version of the WannaCry Ransomware which created an international crisis in 2017[1] (See Figure 3-1).



Figure 3-1

After execution, the local user files were all encrypted and renamed with the .WNCRY extension. The originals were deleted from their respective folders and the “WannaDecryptor” tool of the ransomware startup. The wallpaper was changed as well by the program.

The ransomware provides free decryption for 10 random files.

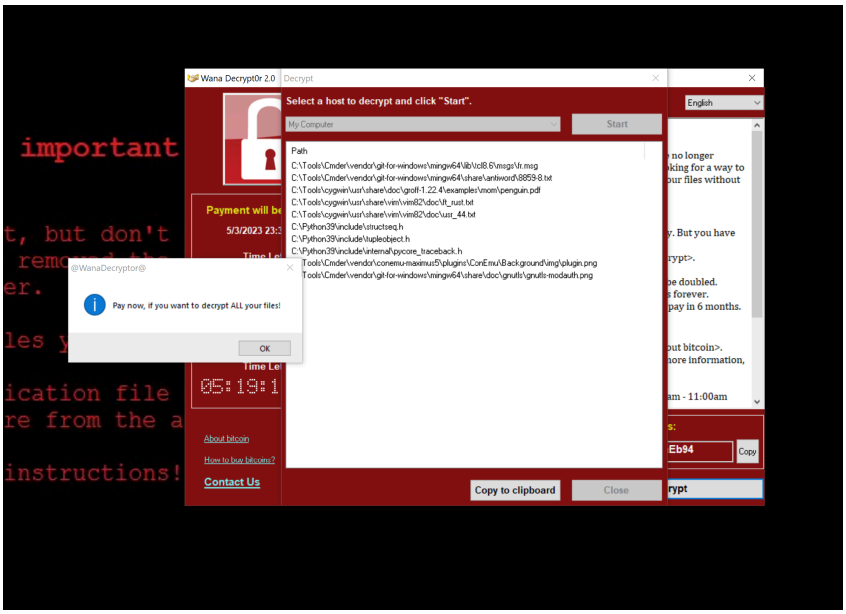
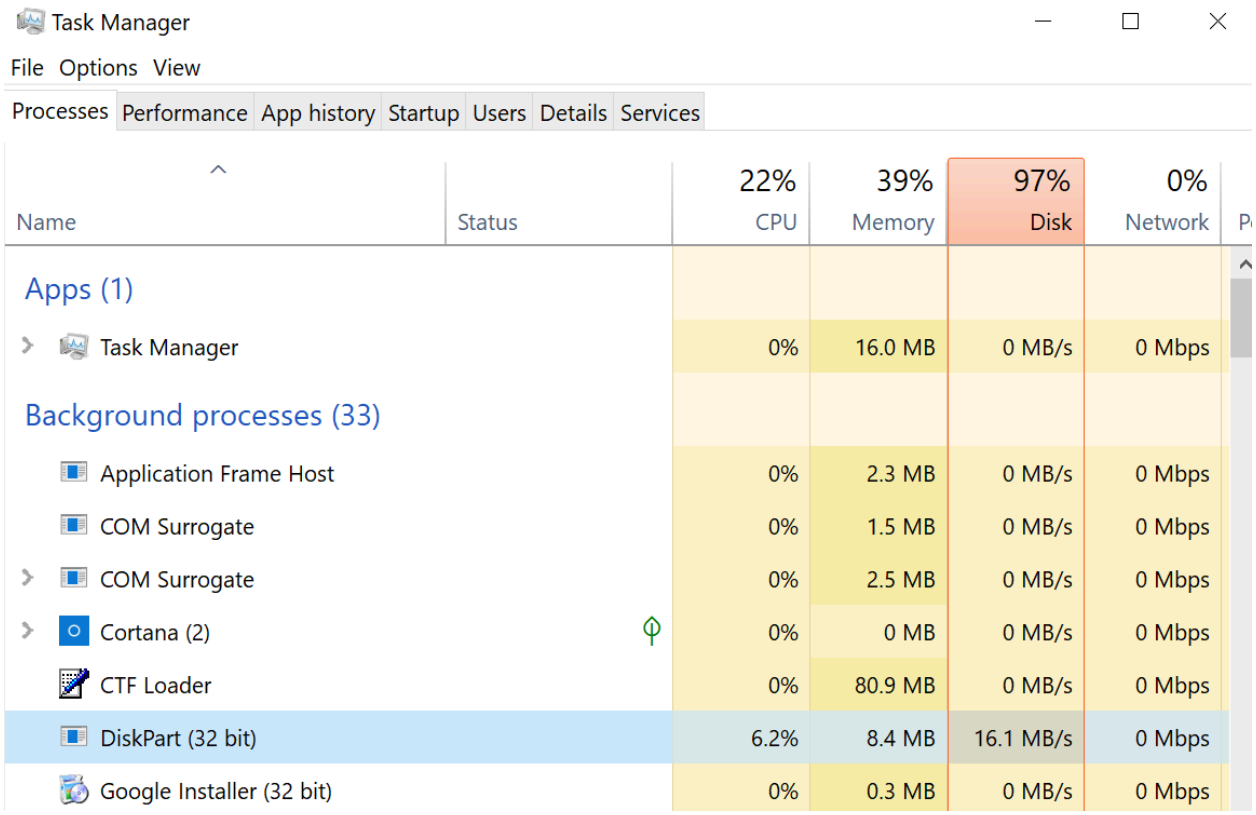


Figure 3-2

The virtual machine during execution was nearly unusable because of unstable performance. An expected high disk usage was reported from Task Manager on Windows, probably due to file encryption and disk scanning routines which are expected to be found inside a ransomware sample. In some instances, CPU usage was extremely high as well, but this was inconsistent during execution.



Due to the machine being extremely slow in this state, only minimal analysis was performed on the sample without manual control of its execution flow by using a debugger.

3.3 Planned Methodology

After observing the results, further analysis was planned and performed.

The decided procedure to be used during the analysis included several techniques including:

- Static Analysis
  - PE File and header Analysis
  - String Analysis
  - Disassembly of the executable files
  - Analysis of any artifact produced by the execution
- Dynamic Analysis
  - Debugging
  - Network Traffic Analysis

Any extracted file from the sample was identified using the “Detect it Easy” tool [3] which will be shortened in this document as DIA.

3.4 Dynamic Analysis

Dynamic Analysis due to the nature of the malware was the one mostly used for both identifying and further analysis of the sample behavior.

Dynamic Analysis involves studying the sample during runtime. During execution, actions taken by the malware can be observed directly while happening. By using debugging software, execution can be stopped and resumed, allowing in-depth observation of the machine’s state in several stages of the malware execution. By using network traffic capturing software, any network-related activity can be tracked and inspected making easier analysis of network functionality.

3.4.1 Debugging

OllyDbg2 was initially the chosen debugger due to its ability to maintain breakpoints saved across sessions and the preinstalled plugin in the provided installation for dumping executable modules to disk.

It was maintained as the main debugging software for the analysis of the base module.

During debugging, the executable would be analyzed during runtime by analyzing its behavior either per instruction or per procedure basis depending on complexity and relevance. Often analyzing the assembly of the executable file inside a debugging environment was preferred due to the presence of runtime information inside the memory. Inside disassembly software, reverse engineering of several parts of the malware would have been much more difficult to perform due to the lack of such information present only during execution.

For instance, obtaining the AES key for decryption of the DLL encryption component would have required more steps and further complexity if it was not found inside memory during execution.

After the DLL encryption component was retrieved, the debugging environment was switched to IDA.

Apart from being easier and more efficient in the loading process of a DLL module inside IDA, the new environment could perform analysis on both the DLL and the base executable at the same time and maintain the original flow of execution and flow of data. Compared to manually calling the DLL export, this allowed a much more efficient environment to reverse and analyze the DLL module.

It also allowed better static analysis of the executable since the disassembler and the debugger were located inside the same environment.

3.4.2 Network Traffic Analysis

Wireshark was the program used for network traffic analysis. Because of time constraints and a low amount of network functionality, network analysis was conducted in a limited capacity.

The content of packets was captured but not inspected enough to have a clear understanding of what is the exact content inside.

3.5 Static Analysis

Static analysis was used mostly as support for dynamic analysis during the initial stages. Originally, it was planned to reverse the base executable in detail and document findings, but it was later decided due to time constraints and complexity to use static analysis as an aid for dynamic analysis which itself provided enough information to document the sample behavior inside the machine.

3.5.1 PE File and Header Analysis

PEStudio was the tool mainly used to analyze the PE executable, its imports, and exports, strings, header, whether the executable was packed or unpacked, whether it was encrypted or decrypted, embedded resources, and other general information.

3.5.2 String Analysis

PEStudio was used for searching strings inside the executable. Hardcoded strings contained information about Windows API calls, as well as other functionality used by the application, but the most important data retrieved was generally data specific to the sample itself.

For instance, the password for an embedded resource file was hardcoded inside the executable and later used for the extraction of the files without sample execution.

PEStudio was the tool of choice for string analysis due to automatic parsing, organization, analysis, and ranking according to suspiciousness.

3.5.3 Disassembly

Disassembly of the executable modules was used in greater amounts for analysis of the encryption DLL. This was because of several reasons. One of them was the switch from OllyDBG2 to IDA as the debugging environment. The choice was made because of being easier to perform debugging of DLL modules inside IDA compared to starting the DLL using LoadDLL.exe with OllyDBG2 attached. IDA not only supported instant loading and automatic setup of the execution environment for the DLL module but it was possible as well to perform debugging of it alongside the original base executable. This allowed maintenance of the original flow of execution and saved the trouble of manually calling the export with the ordinal with the possibility of the DLL not behaving as originally intended.

Disassembly was more prominent with the DLL because once it was loaded in IDA both debugging and disassembly were possible in the same environment and changes in one side of the workflow were directly reflected in the other.

3.5.4 Analysis of artifacts produced by the execution

Any file produced by the executable was attempted to be identified and inspected as much as possible.

Identification was performed using the DIA tool when needed. If the content was found to be readable by any software package installed, it was inspected for further identification.

3.6 History of procedure

In this section, a general chronological overview of what actions were taken during analysis is listed with only a minimal mention of results and technical aspects of the procedure.

This is needed to explain why actions were performed based on which decisions and the factors those decisions were based on.

3.6.1 Initial Inspection

During preliminary analysis, two stages were performed to gather initial information about the sample executable.

3.6.1.1 PE File and Header Inspection

During this stage, as previously mentioned, PEStudio was used for obtaining initial information about the sample PE executable file. The information found regarding imports and libraries used was useful to gather general ideas on what was to be expected once more in-depth analysis was performed, but, independently, they did not provide enough knowledge about the inner architecture of the sample being analyzed.

The evidence retrieved of the most value was the embedded password-protected ZIP file which was found to be embedded inside the executable.

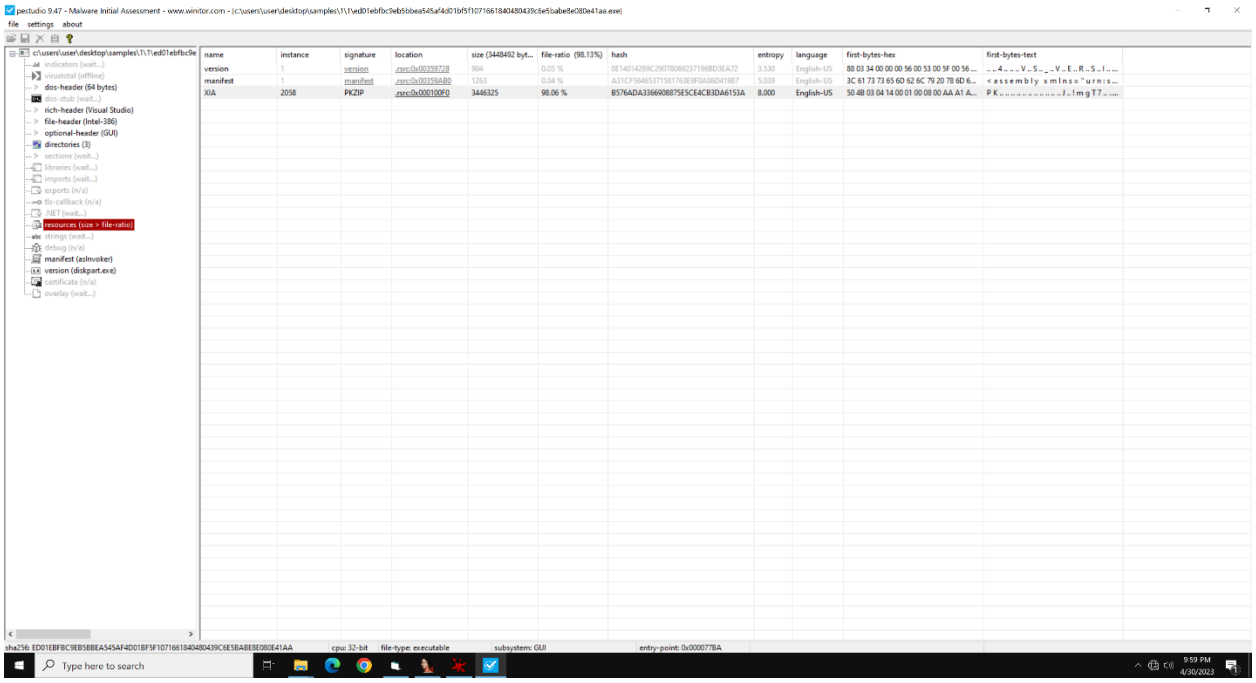
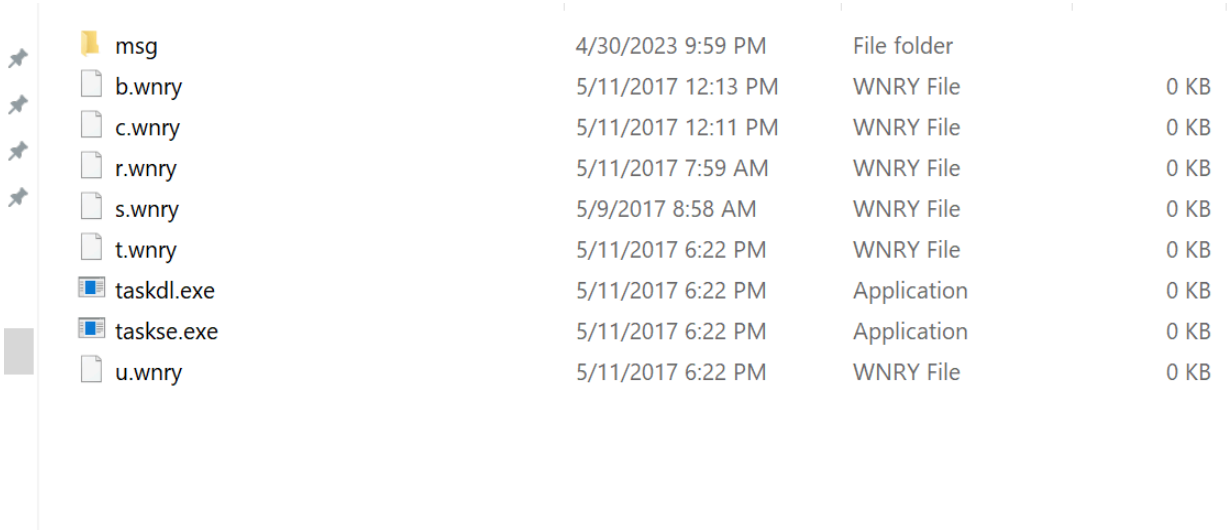


Figure 3-3

At this stage, no password was possessed for decompressing the files inside, but 7-Zip was used to observe the files contained by the archive.



msg	4/30/2023 9:59 PM	File folder	
b.wnry	5/11/2017 12:13 PM	WNRY File	0 KB
c.wnry	5/11/2017 12:11 PM	WNRY File	0 KB
r.wnry	5/11/2017 7:59 AM	WNRY File	0 KB
s.wnry	5/9/2017 8:58 AM	WNRY File	0 KB
t.wnry	5/11/2017 6:22 PM	WNRY File	0 KB
taskdl.exe	5/11/2017 6:22 PM	Application	0 KB
taskse.exe	5/11/2017 6:22 PM	Application	0 KB
u.wnry	5/11/2017 6:22 PM	WNRY File	0 KB

Figure 3-4

Because of the observed results during the general test, it was assumed the zip folder contained both assets used by the decryption software displayed to the user, as well as other configuration files needed for setting up the ransomware on the victim machine. Two executables seemed to be included as well, but their purpose was still unknown.

3.6.1.2 String Analysis

In the same session of PEStudio used for PE file and Header inspection, string analysis was performed. Many strings of interest were found, which were useful to understand more of what was to be expected by further analysis. Hardcoded file extensions suggested the sample encryption routine was performed on files with a predefined set of extensions.

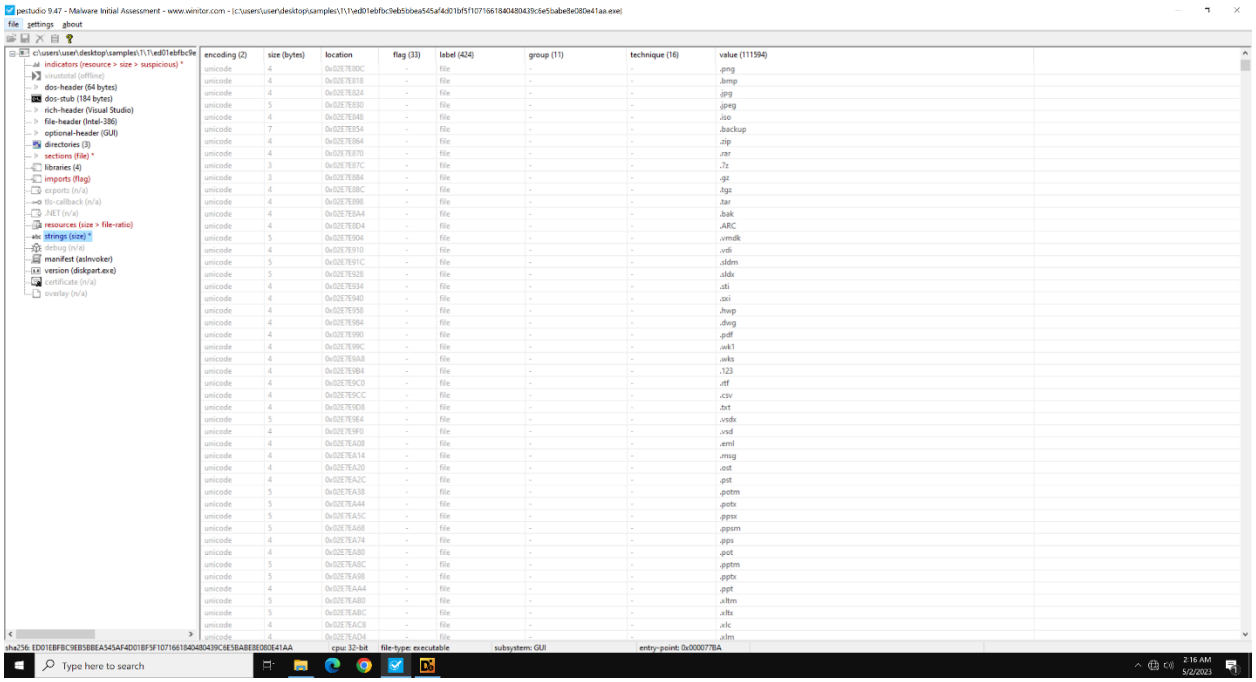


Figure 3-5

Hardcoded file names of files found inside the dumped ZIP resource were expected. Bitcoin addresses were found as well listed among the strings.

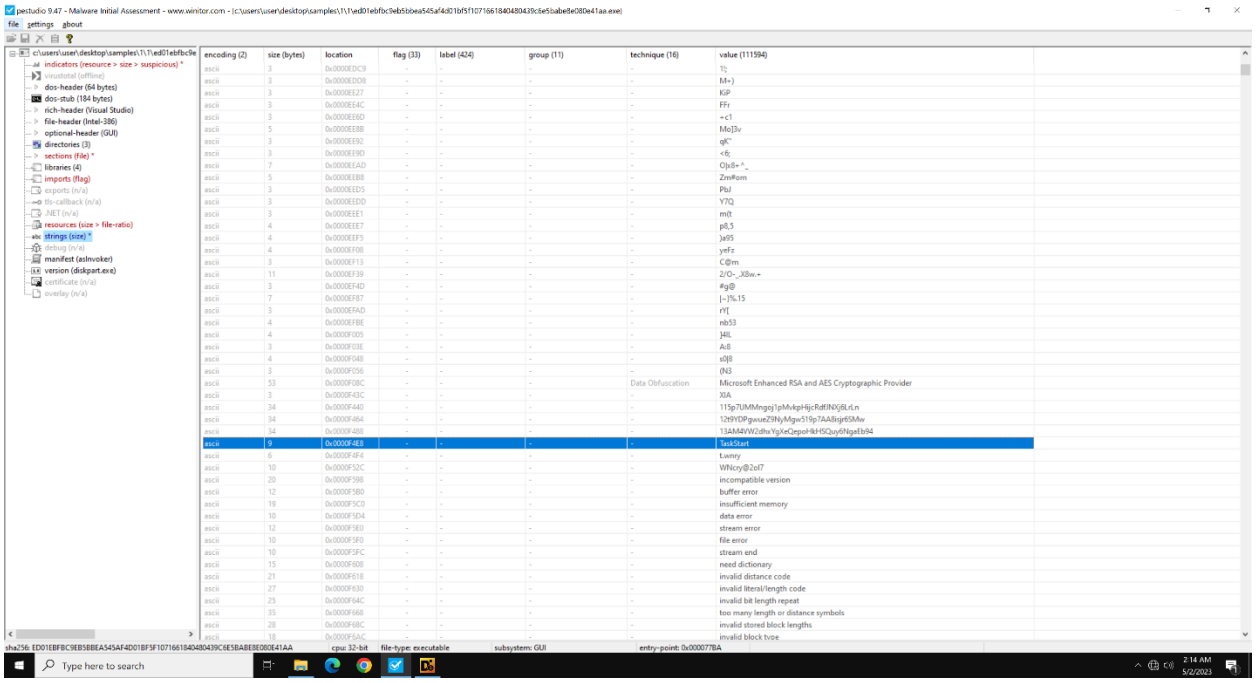


Figure 3-6

This and other information involving Windows API calls again, were not new at this stage due to the previously obtained information but some new information was found and deemed to be useful for later stages of the analysis.

3.6.2 Base Executable Analysis

After PEStudio was used to gather information about the executable, a first attempt at analyzing statically the sample main executable was made by using IDA disassembler. However, due to initially finding it difficult to understand the data flow inside the application only possible to be retrieved during runtime, it was switched to a dynamic approach by using OllyDbg2. IDA was kept open for reference in case it was needed to keep track of certain sections of code found to be of interest.

3.6.2.1 Debugging

During debugging, the general behavior was observed inside the executable. Several functions were found to be difficult comprehension by using only the debugger due to the textual representation of the code disassembly not always being clear enough. Code contained many functions which were not related to the Windows API, so all of them were unnamed and many times not of easy understanding

IDA and its graphical representation of execution flow was used as aid for reversing those routines better.

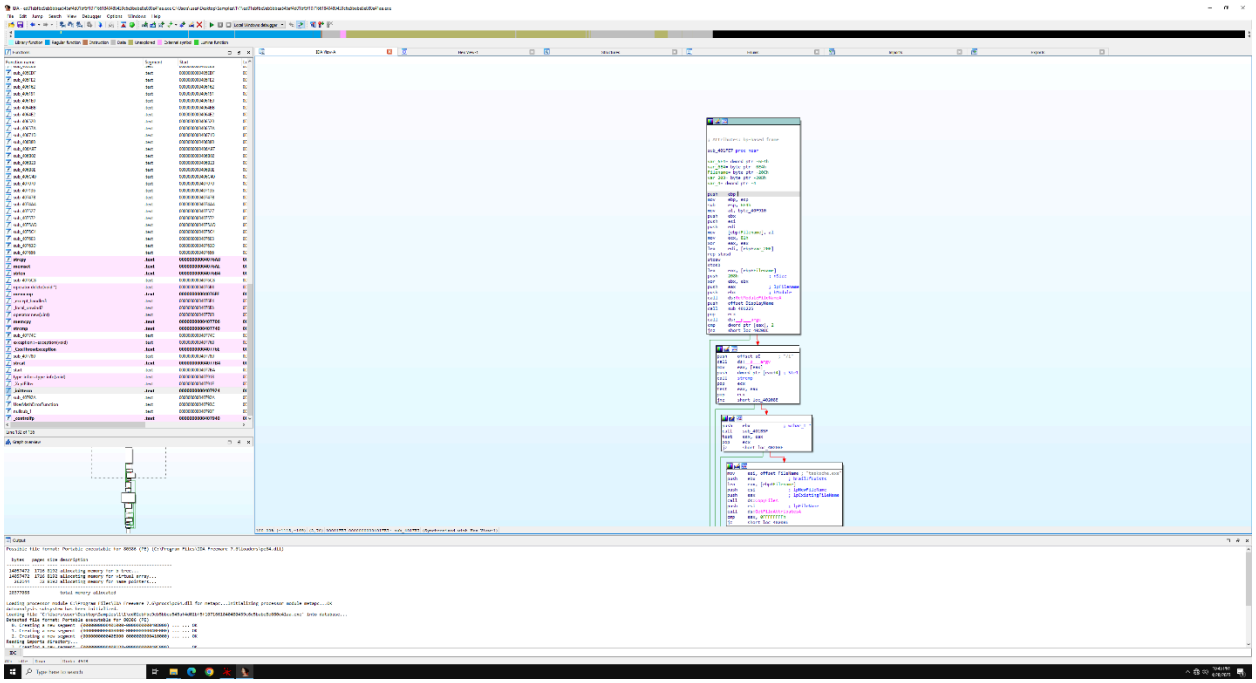


Figure 3-7

While most of the functionality of the executable was understood some was not and analysis on it was left unfinished due to time constraints it was left marked as future work.

Results obtained from debugging allowed a reconstruction of most of the activity performed by the sample on the machine, as well as provided knowledge regarding the existence of a second component, a DLL module, which was found to be hidden among the files contained by the previously found XIA ZIP archive. The DLL was invoked using the “TaskStart” export as can be observed in Figure 3-8.

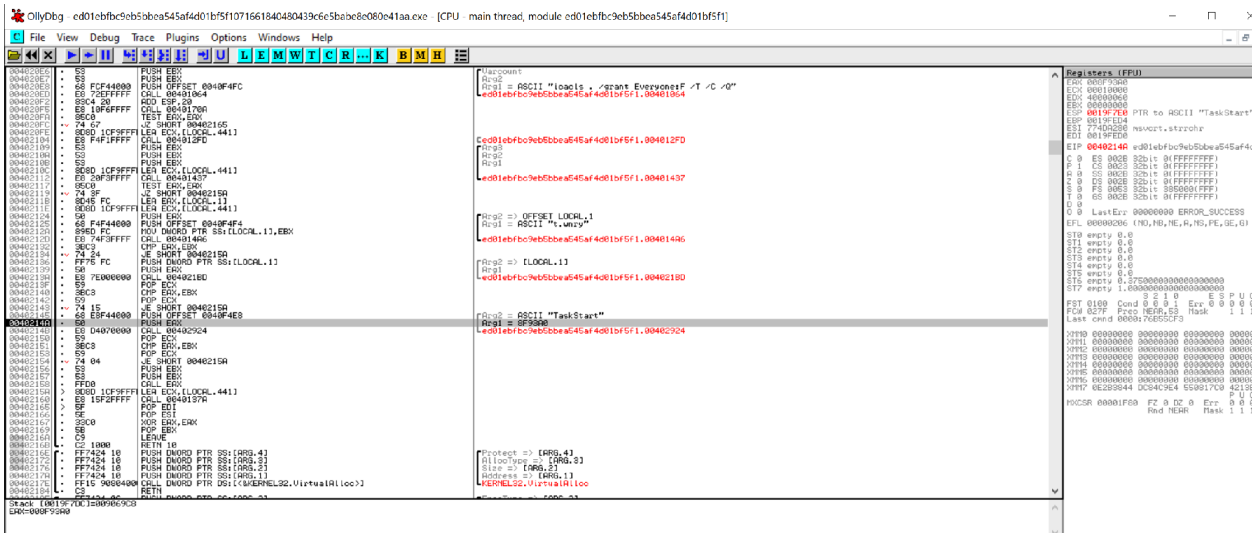


Figure 3-8

It was unknown at this stage how analysis could be performed on such a module. It was attempted to get it dumped from memory, but inexperience and lack of information on the Internet did not allow its retrieval.

However, it was known that the DLL was loaded and executed after performing operations on the t.wnry file, involving cryptography. This suggested that either the t.wnry file was an encrypted archive or the DLL itself was encrypted.

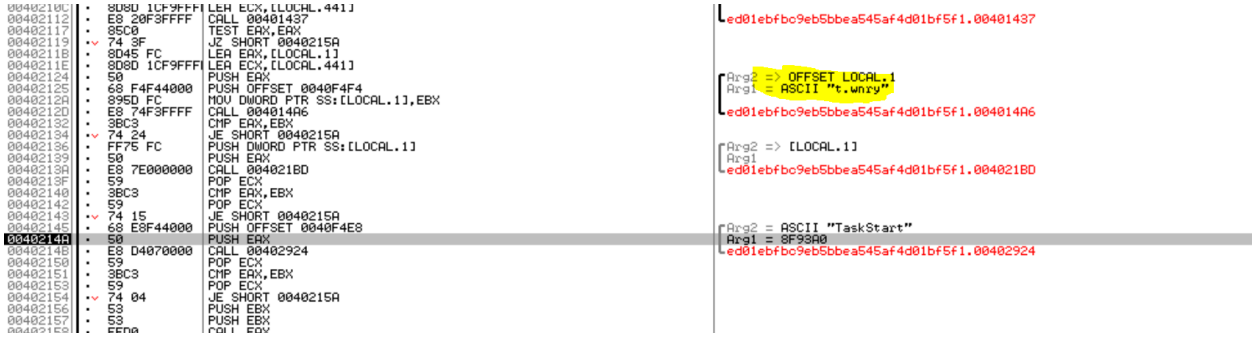


Figure 3-9

It was also known that the DLL was responsible for encrypting user files.

This was observed when the call to the “TaskStart” export instantly performed encryption of files alongside extraction of new files unique to that DLL.

Those files were chosen not to be analyzed until it could be obtained the original file and have the ability to perform analysis on it.

This stage also allowed the recovery of the ZIP-protected archive password, a string previously found to be hardcoded inside the executable,” WNcry@2ol7”, which allowed extraction and later analysis of the files used by the sample during execution without having to mandatorily execute the sample itself.

3.6.2.2 Output Files Inspection

Several files were extracted by the XIA archive from the sample during runtime. All of them at this stage were attempted to be analyzed.

msg	5/2/2023 5:04 AM	File folder	
b.wnry	5/11/2017 8:13 PM	WNRY File	1,407 KB
c.wnry	5/11/2017 8:11 PM	WNRY File	1 KB
ed01ebfbc9eb5bbea545af4d01bf5f10716...	5/14/2017 3:29 PM	Application	3,432 KB
r.wnry	5/11/2017 3:59 PM	WNRY File	1 KB
s.wnry	5/9/2017 4:58 PM	WNRY File	2,968 KB
t.wnry	5/12/2017 2:22 AM	WNRY File	65 KB
taskdl.exe	5/12/2017 2:22 AM	Application	20 KB
taskse.exe	5/12/2017 2:22 AM	Application	20 KB
u.wnry	5/2/2023 10:34 AM	WNRY File	0 KB

Figure 3-10

The DIA tool was used for identifying any file which used a known file format. (See Appendix A—4.3.1). The ones which did not were attempted to be opened using a Hex editor. In this case, the machine came preinstalled with the 010 editor. Most of the files as previously correctly assumed were assets used by the decryption tool. Specifically, most of them were instructions on how to pay and decrypt the files.

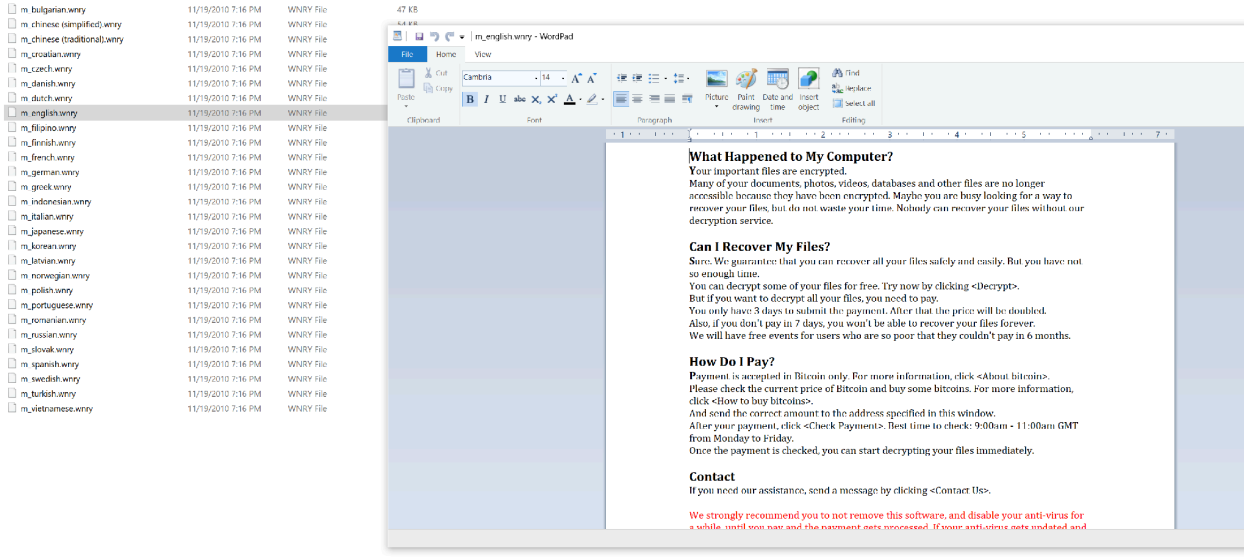


Figure 3-11

Inside the “msg” folder several versions of the information in RTF format, each in a different language were found. Even if the decryption tool was not reversed, from the GUI of the program during the general test, it was clear that these are loaded by it when the dropdown menu specifying the language is

set to a different one. What remained unclear to the very end, is where the msg folder containing the RTF files with the instructions was moved to. Due to time constraints, major focus on research on how to decrypt the DLL module, and this aspect not being largely useful for analysis of the sample it was decided to mark it as future work to be performed on the sample alongside reversal of the decryption software.

While only two executables were expected, it was found the file “u.wnry” was a PE executable as well. Specifically, the executable was found to be the decryption tool later executed and displayed to the user.

As mentioned, the decryption tool was not analyzed but marked as future work that could be performed on the sample.

The t.wnry file previously found to be involved in the loading of the DLL module was attempted to be loaded inside the hex editor. The file had the first 8 bytes used by the string “WANNACRY!”, while the content inside seemed to be unknown. Since it was desired to analyze the DLL module as there was a high probability of leaving without analyzing a significant part of the sample, it was decided afterward to attempt its decryption and later have it analyzed separately.

The two executables “taskse.exe” and “taskdl.exe” were not analyzed as the focus was shifted majorly towards the DLL module decryption and analysis.

Since those two steps required a significant amount of time, the two executables were marked as future work to be performed on the sample. Their mention was later found during the analysis of the DLL module, but their exact purpose was not understood.

The c.wnry file was found to be a collection of Onion addresses and a Tor download link.

This suggested that the sample had some network activity performed during execution.

Proof of Tor being used was supported as well by the “s.wnry” file which was revealed to be an archive containing an installation of Tor.

The link was assumed to be used in case this file was corrupted. Tests during network traffic analysis were scheduled regarding the voluntary corruption of this file. If this file was found to be corrupted by the sample, and an HTTP request was performed to download the Tor browser, network traffic from such activity would have surely been intercepted by Wireshark.

Due to time restrictions and Wireshark not working as expected during networking tests, this latter aspect was left without being analyzed.

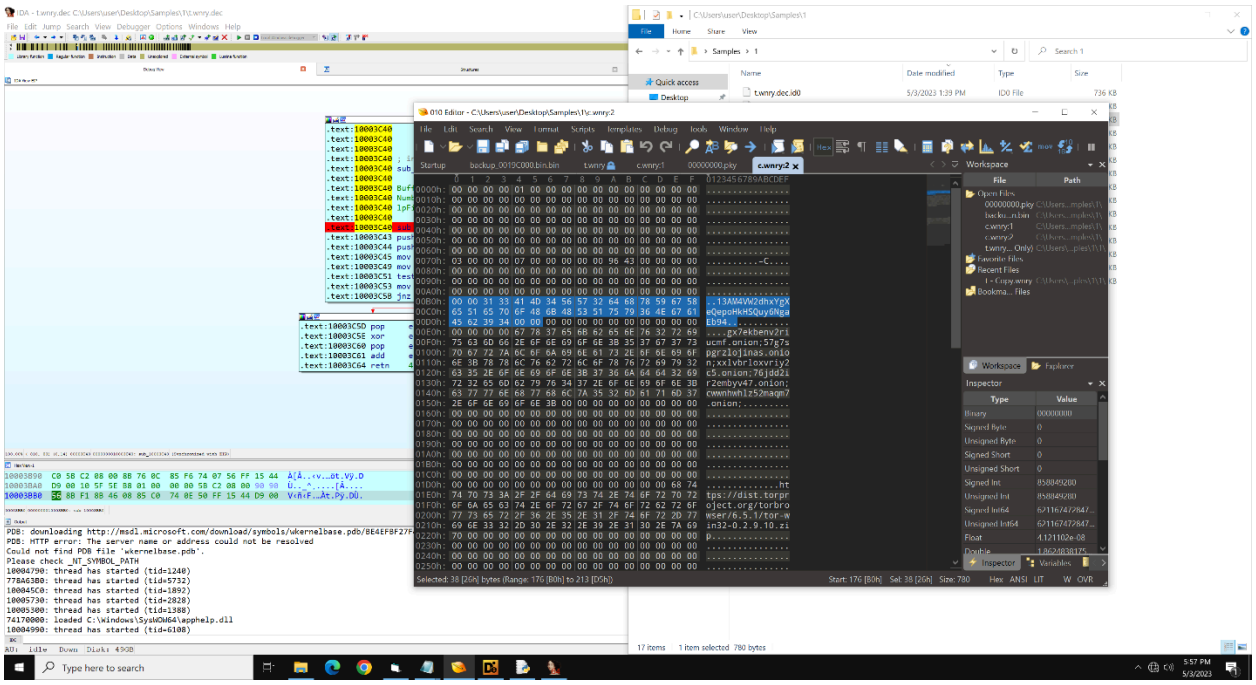
It was left to be inserted instead among future work to be performed on the Sample.

Inside the “c.wnry” file it remained unknown the purpose of a pair of characters, “-C”. While it was assumed they could be a command line parameter for Tor, its documentation proved the opposite.

Because of time constraints and the aspect of not being too important for the analysis, it was marked among the other future work that could be performed on the sample.

During debugging and disassembly, even if it was not spent a consistent amount of time checking code involving the “c.wnry” file, it was found that the file was accessed both in read and write mode.

Comparing a copy of the file during and before execution revealed that the file was filled before the loading of the encryption DLL with one of the hardcoded Bitcoin addresses inside the executable.



This meant that the c.wnry file was not used only by the base executable but as well by the decryption tool and possibly it was assumed by the DLL as well. This latter assumption was since at that moment it was not found yet code regarding the startup of Tor so it might have been possible its execution was handled by the DLL rather than the base executable.

After analysis of the artifacts left by the base executable, the shift was mainly moved toward decryption of the DLL from the t.wnry file.

3.6.3 Decrypting DLL module

This step involved a large amount of time and limited further analysis of previously unfinished work and later analysis.

The high complexity found in decrypting the DLL was the limited amount of information on the file at hand, “t.wnry”.

It was understood only after a significant amount of research that the file header structure was identical to the “.WNCRY” files, the encrypted version of the user files, left by the ransomware.

The first 12 bytes were identical using the same “WANNACRY!” string followed by “00 01 00 00”.



Figure 3-12

While what came afterward was different in t.wnry and WNCRY files used for comparison, it was found a section full of NULL bytes in the same location as well (See Figure 3-13 and Figure 3-14 for reference).

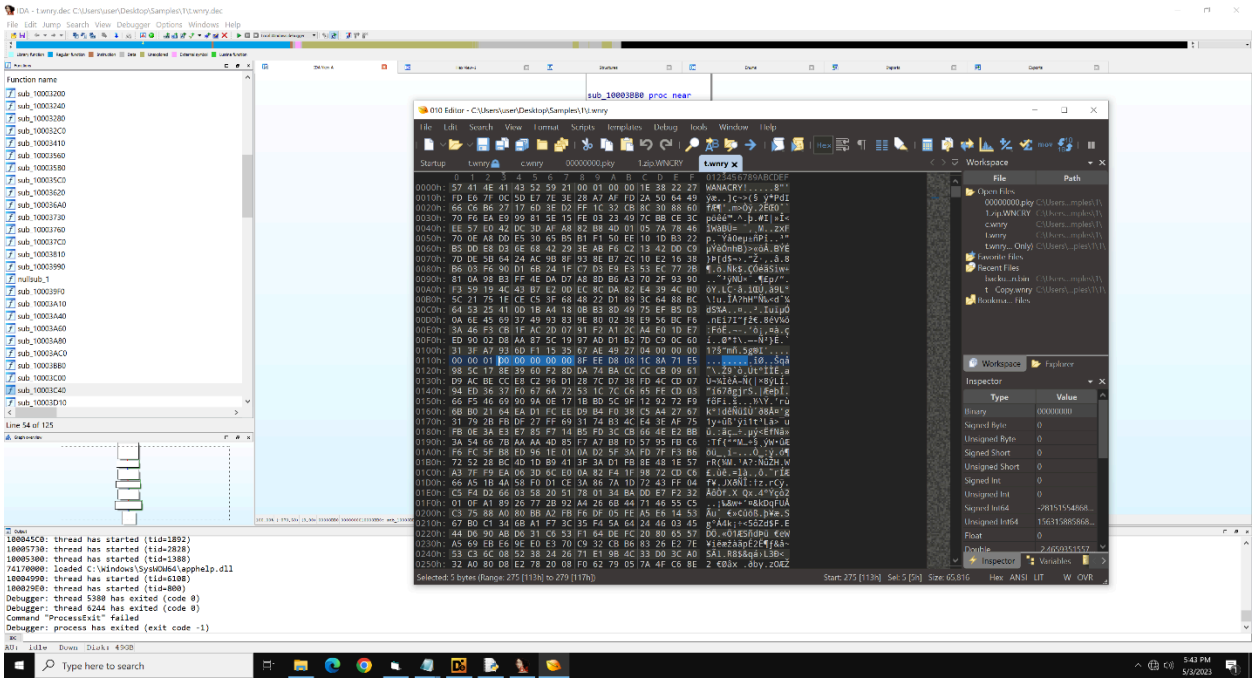


Figure 3-13 Null bytes inside the t.wnry file

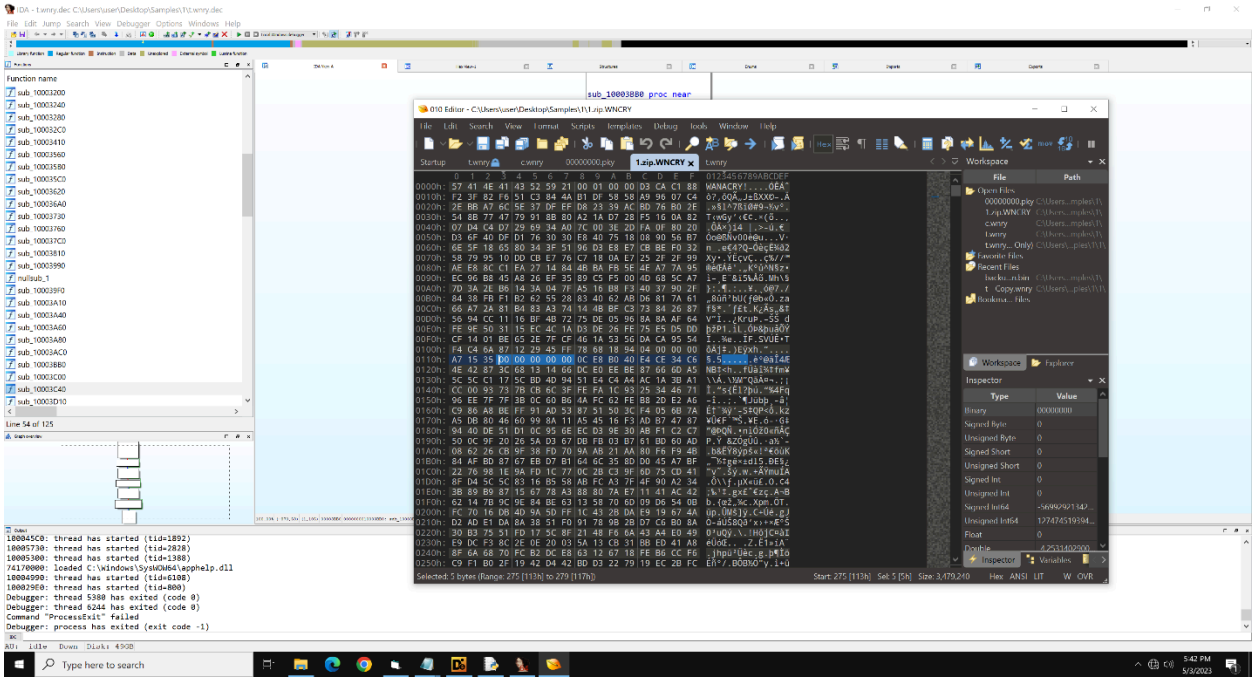


Figure 3-14 Null Bytes inside WNCRY encrypted file format

It was theorized at that moment of the probability of it being the end of the header section and the rest being the encrypted file itself. Inside the header, it was unknown at this stage what could be the content, but it was speculated that the software used a common format for encrypting files.

However, at this point, no knowledge was possessed about the encryption used for this file.

After a significant amount of time spent inside the debugger monitoring the cryptographic routines called before the execution of the DLL module, it was found that the routines responsible for decrypting the DLL had a pointer being used consistently which pointed at a specific location in memory.

While for a significant amount of time, the content at this location remained of unknown purpose, and the code using it was not found to be easily reversible, it was noticed that a pair of 16 bytes were dumped by a specific function of the executable during the execution of the cryptographic routines.



Figure 3-15

Those bytes were of a fixed value and always dumped at the same location. Another pair of 16 bytes were dumped as well right after the one just mentioned.

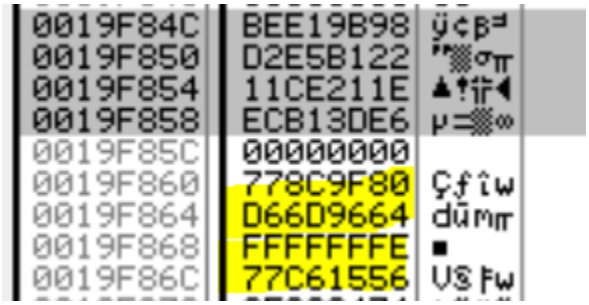


Figure 3-16

Since cryptography was involved it was thought that maybe the two were pairs composed of a private and public key. However, it was an option quickly discarded since the observed key length of 128 bits is not secure in asymmetric encryption.

Since no other concrete information was available for decrypting the module, research was conducted about the two pairs of 16 bytes using external resources.

While public key cryptography was excluded because of key length, there was no reason why the two pairs were involved instead of symmetric encryption.

AES-128, a common and secure symmetric cipher, had for example the right length to match the pattern in memory.

What remained unclear is why 2 AES keys would be generated rather than just one. Since those encryption routines involved the decryption of the DLL file inside t.wnry, only one key would have been enough.

Since at this stage, there was confusion in the obtained data, and doubt was possessed about the usefulness of it, it was decided instead to perform an online search about the two strings.

Since the sample is a version of a popular malware, if these two pairs of bytes had any relevance, they were sure to be mentioned in malware databases.

While the second pair of 16 bytes was not referenced, the first was confirmed in several online databases and malware analysis websites to be the key for decrypting the DLL. It was found as well that the key itself was originally stored in the t.wnry file itself at a fixed location and decrypted using a hardcoded RSA key.

Since already enough time was spent attempting to decrypt the DLL, it was decided to be enough for the retrieval of the AES key from memory. Recovery of the RSA key was marked as future work that could be performed on the sample.

However, from the recovered information, it was at this stage clear that the common file format used for encrypted files and the DLL itself had inside its header the AES key used for the decryption and encryption of the files. However, the files themselves had their AES key unrecoverable due to being encrypted with a public key where the corresponding private key is supposedly possessed by the creator of the sample.

A Python script using the Cryptography library for decrypting the DLL was created and used to decrypt the DLL module and proceed with the analysis (See Appendix B).

3.6.4 Encryption DLL Analysis

The DLL, unlike the base executable, could not be launched in standalone mode. Rather than using LOADDLL.exe and manually calling the export “TaskStart” by using OllyDbg2 it was found that IDA was much more suitable for loading DLL modules.

Its ability to debug both the execut able and the DLL at the same time in the same environment was found to be a much more optimal workflow.

It also allowed static analysis and dynamic analysis in the same environment making the work environment more efficient. Also, the output of IDA for the DLL module was found to be of high quality and easy enough to comprehend many times without having to perform execution and debugging of the sample.

3.6.4.1 PE File and Header Analysis

Before debugging, as for the base executable it was chosen to perform a PE file and header analysis using PEStudio. Reference to functions like FindFirstFile or FindNextFile from the Windows API was expected to be found as this was known to be the component responsible for the encryption of user files.

Not too much information was found to be of extreme use but like the base executable the information retrieved from PE studio allowed a much better understanding of what was to be expected once more in-depth analysis was performed on the sample.

3.6.4.2 String Analysis

Several strings of interest were found for the DLL module and several of them included important information about the activity of the sample on the machine.

A list of file extensions was found inside the strings section of the DLL as well making it unclear why there is a duplicate in the base executable if no encryption of files is performed there.

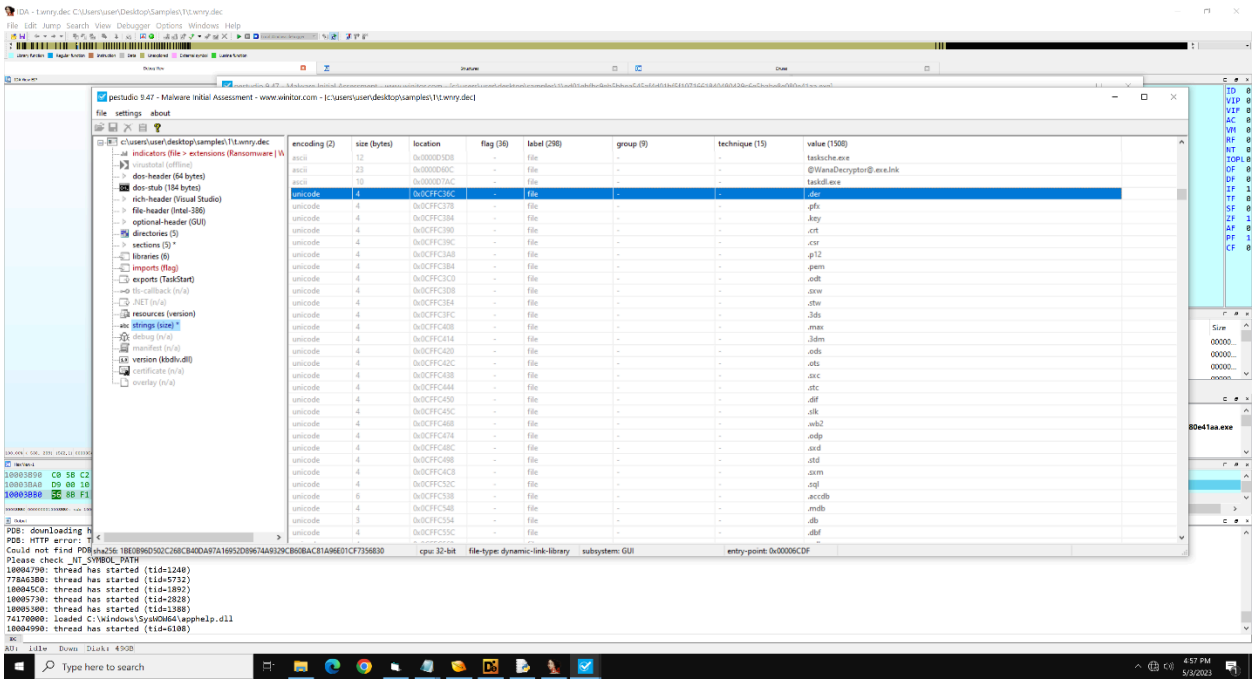


Figure 3-17

This was deemed not to be of primary importance so was left among the future research that could be performed on the sample.

Some of the important information discovered was, for example, a list of several CMD commands that terminated several database processes usually found in servers such as MySQL. This probably was

executed during runtime to impede administrators from performing backups of the database from memory to a remote destination.

-	del /a %%0
Command-Line Interf...	cmd.exe /c reg add %s /v "%s" /t REG_SZ /d "\"%s\"" /f
Hidden Files and Direc...	attrib +h +s %C:\%s
Command-Line Interf...	cmd.exe /c start /b %s vs
Disabling Security Tools	taskkill.exe /f /im mysqld.exe
Disabling Security Tools	taskkill.exe /f /im sqlwriter.exe
Disabling Security Tools	taskkill.exe /f /im sqlserver.exe
Disabling Security Tools	taskkill.exe /f /im MSExchange*
Disabling Security Tools	taskkill.exe /f /im Microsoft.Exchange.*

Figure 3-18

3.6.4.3 Initial Disassembly

The encryption DLL was found to be generally easier to analyze statically compared to the base executable.

Due to this, disassembly and static reverse engineering were performed as initial analyses instead of directly attempting debugging on the DLL module.

During this stage, an initial chronological reconstruction of some of the activities performed by the sample was created.

In some sections, difficulty was found in assessing sample behavior exclusively by using static analysis, afterwards, they were attempted to be cleared out during debugging of the DLL module.

During the disassembly of the DLL module, it was found that encryption routines and other functions carried out by the executable were using threads during a significant part of its execution.

```
.text:10005C96 mov     eax, [esi]
.text:10005C98 push    1
.text:10005C9A mov     ecx, esi
.text:10005C9C call    dword ptr [eax]
.text:10005C9E mov     esi, ds:CreateThread
EIP .text:10005CA4 push    ebx                             ; lpThreadId
.text:10005CA5 push    ebx                             ; dwCreationFlags
.text:10005CA6 push    ebx                             ; lpParameter
.text:10005CA7 push    offset sub_10004790             ; lpStartAddress
.text:10005CAC push    ebx                             ; dwStackSize
.text:10005CAD push    ebx                             ; lpThreadAttributes
.text:10005CAE call    esi ; CreateThread
.text:10005CB0 mov     ebp, ds:CloseHandle
.text:10005CB6 cmp     eax, ebx
.text:10005CB8 jz      short loc_10005CBD
.text:10005CBA push    eax                             ; hObject
.text:10005CBB call    ebp ; CloseHandle
.text:10005CBD
.text:10005CBD loc_10005CBD:                           ; CODE XREF: TaskStart+1D8↑j
.text:10005CBD mov     edi, ds:Sleep
.text:10005CC3 push    64h ; 'd'                       ; dwMilliseconds
.text:10005CC5 call    edi ; Sleep
.text:10005CC7 push    ebx                             ; lpThreadId
.text:10005CC8 push    ebx                             ; dwCreationFlags
.text:10005CC9 push    ebx                             ; lpParameter
.text:10005CCA push    offset sub_100045C0             ; lpStartAddress
.text:10005CCF push    ebx                             ; dwStackSize
.text:10005CD0 push    ebx                             ; lpThreadAttributes
.text:10005CD1 call    esi ; CreateThread
.text:10005CD3 cmp     eax, ebx
```

Figure 3-19

3.6.4.4 Debugging

During debugging of the DLL, activities of the sample as in the base executable were observed.

Thanks to debugging, the activity reconstruction performed during the disassembly of the DLL module was improved. However, several parts of the DLL module were not finished to be analyzed due to time constraints. Those unfinished sections were marked as future work to be performed on the sample.

3.6.4.5 Output files Inspection

The DLL produced several files as output in the working directory as the base executable did. Some of them had their purpose reversed, while others had not.

Files reversed included a VBS script which created a shortcut on the desktop for the decryption tool, or the file containing the public key used to encrypt the randomly generated AES key contained inside the header of each encrypted file.

3.6.5 Network Traffic Analysis

3.6.5.1 Wireshark capture of normal traffic flow

Wireshark was used to monitor ongoing traffic. The machine displayed during the execution of the sample traffic involving ports used commonly by the Tor service.

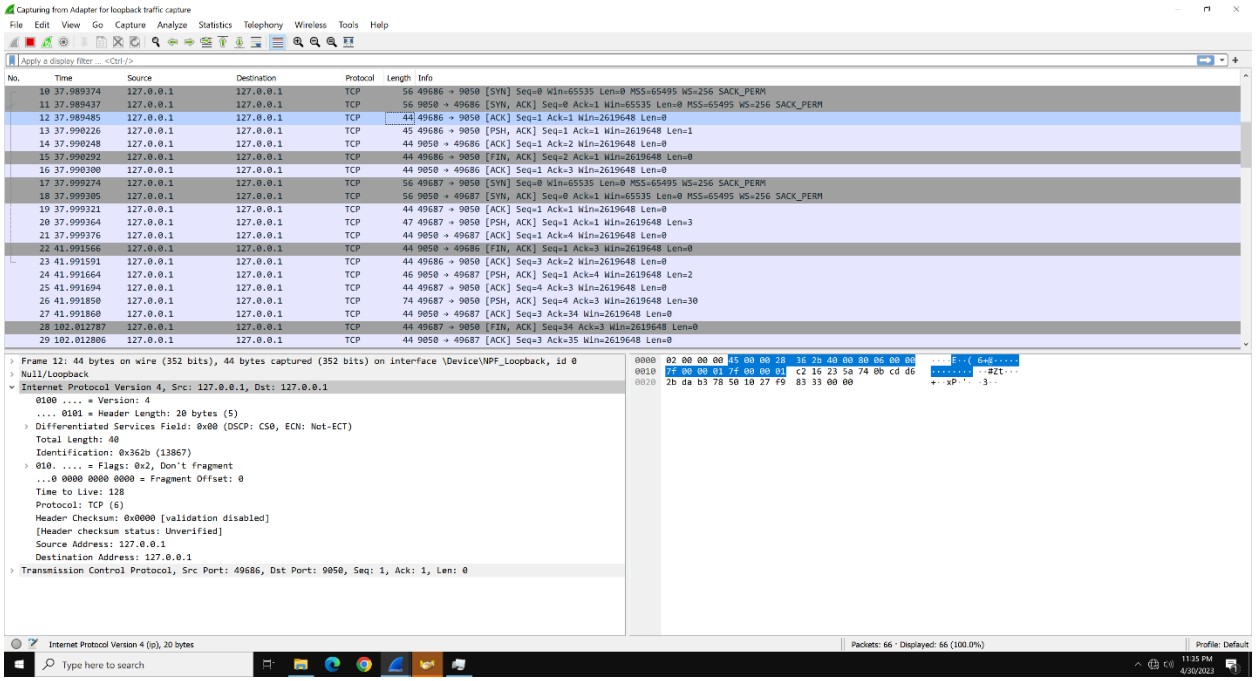


Figure 3-20

A superficial analysis of the data being sent was conducted, but due to time constraints and failure in finding an approach to reverse the content, it was marked as unfinished work which could be performed in the future on the sample.

3.6.6 Attempt at intercepting HTTPS request for Tor bundle download

Because of the previously observed link found inside c.wnry, the theory in which the sample would have used such a link for download of the Tor bundle in case of corruption was attempted to be tested.

This involved manual corruption of the archive where the included Tor installation was compressed after pausing execution right after the decompression of the files inside the resource folder.

However, Wireshark stopped capturing traffic unexpectedly and results from the previous session as well as any HTTPS request were not detected by the tool. Several attempts at restoring old snapshots of the machine were made and several attempts at intercepting network traffic as well.

However, Wireshark was unable to detect any network traffic for unknown reasons.

Due to time constraints, the test had to be marked as future work that could be performed on the sample.

4 Results

After finishing the procedure, results were collected, and a general timeline of the actions performed by the executable was attempted to be reconstructed. While a significant part of the activity was reconstructed, analysis of several parts of the executable remained unfinished often due to time constraints while sometimes time was combined with other factors such as complexity or relevance of any results by the section to be analyzed. As in the case of the Decryption tool, the results that might have been obtained would have not been of key use in recreating the activity of the sample compared to other components.

While several steps performed by the two components of the sample analyzed, the base executable and the encryption DLL, are missing from the activity reconstruction listed below, the key actions that were found are listed below in chronological order, providing a general overview of what the effects of the sample executed are on the victim machine.

4.1 Base Executable Analysis

4.1.1 Activity Reconstruction

The sample generalized flow of execution inside the base executable follows these steps:

- First it loads the password-protected resource in PKZIP format embedded inside the executable
- Decompresses the data using the password “WNcry@2ol7@.
- It reads the configuration c.wnry file and then writes back into it randomly one of the hardcoded Bitcoin addresses
- Then it executes two actions:
    - Executes the command icals to grant itself full read and write permissions to system files
    - Assigns the hidden attribute to the working directory of the sample.
- After doing that it starts decryption of the DLL module located inside the t.wnry file by using the hardcoded RSA key to decrypt the AES key inside its header. The AES key is then used to decrypt the DLL itself
- After decryption the base executable loads the export “TaskStart” from the DLL module to initiate encryption of files.

4.2 Encryption DLL Analysis

4.2.1 Activity Reconstruction

After being loaded into memory and called using the “TaskStart” export, the DLL performed the following actions:

- Creation of the mutex MsWinZonesCacheCounterMutexA

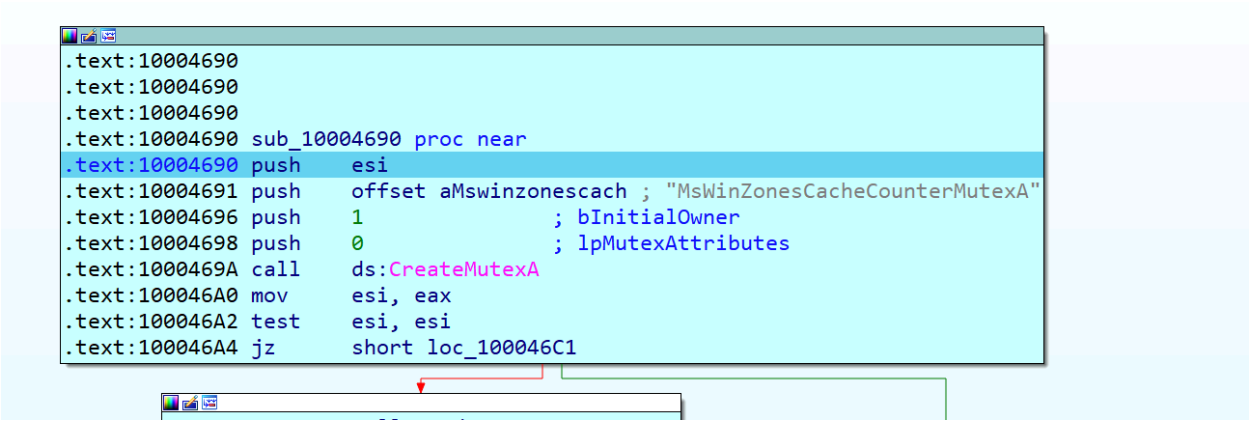


Figure 4-1

- Switching the working directory to the path of the executed module
- Push offsets into the stack of three buffers



Figure 4-2

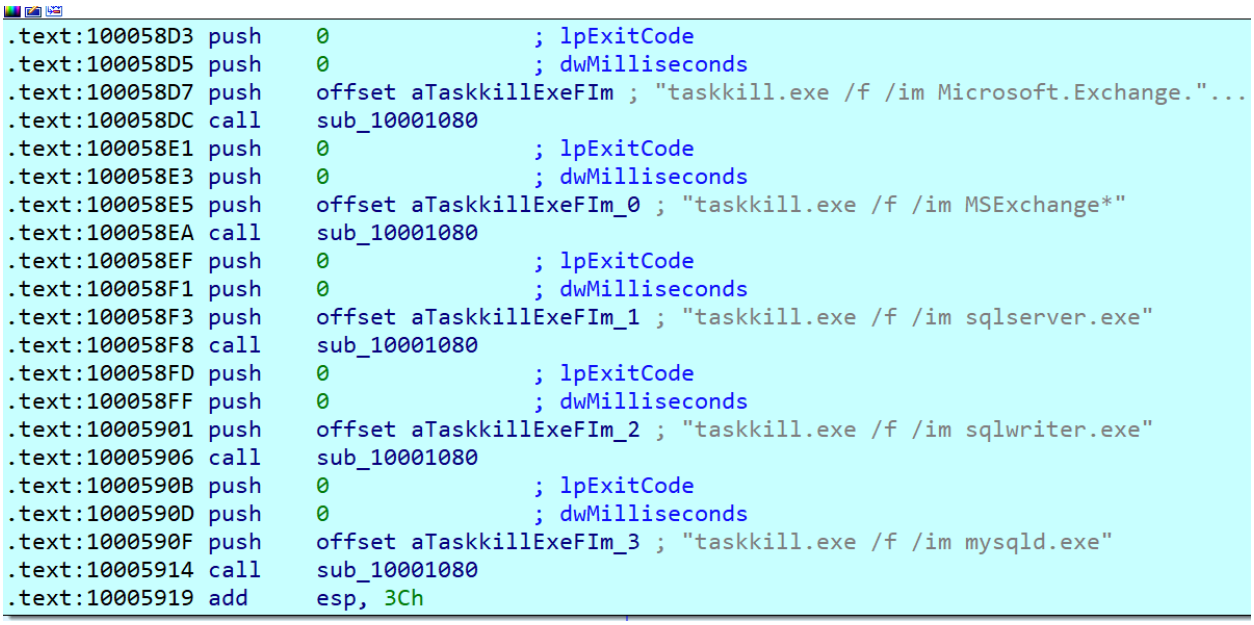
- Push offsets into the stack of several Windows API functions involving both files and cryptographic routines



Figure 4-3

- Reads contents of the c.wnry file after opening the file using fopen() in read mode
- Creates three files:
  - 00000000.pky – Public Key
  - 00000000.eky – Encrypted Private Key
  - 00000000.res – Unknown purpose
- Checks for the presence of the file 00000000.dky file (Decrypted Private Key)
- It generates a key pair and exports the public RSA key to the 00000000.pky file
- The private key is encrypted with a hardcoded RSA public key and stored in the 0000000. key file
- A thread is spawned which checks for logical drives attached to the system
- Another thread is spawned which is responsible for the encryption routine. To do so:
  - It iterates each directory on the system

- using Windows API functions FindNextFile or FindFirstFile when changing the directory it parses the next user file to encrypt (Avoiding system directories, probably to avoid corruption of the machine)
- It generates using the CryptGenRandom an AES key which is then used to encrypt the file
- It then encrypts the keys using the public key previously generated and stores it inside the WNCRY file, which contains both the key and the encrypted user file.
- The code at a specific section also attempts to kill several database software by using Windows system commands as seen in Figure 4-4



```
.text:100058D3 push    0               ; lpExitCode
.text:100058D5 push    0               ; dwMilliseconds
.text:100058D7 push    offset aTaskkillExeFIm ; "taskkill.exe /f /im Microsoft.Exchange."...
.text:100058DC call    sub_10001080
.text:100058E1 push    0               ; lpExitCode
.text:100058E3 push    0               ; dwMilliseconds
.text:100058E5 push    offset aTaskkillExeFIm_0 ; "taskkill.exe /f /im MSExchange*"
.text:100058EA call    sub_10001080
.text:100058EF push    0               ; lpExitCode
.text:100058F1 push    0               ; dwMilliseconds
.text:100058F3 push    offset aTaskkillExeFIm_1 ; "taskkill.exe /f /im sqlserver.exe"
.text:100058F8 call    sub_10001080
.text:100058FD push    0               ; lpExitCode
.text:100058FF push    0               ; dwMilliseconds
.text:10005901 push    offset aTaskkillExeFIm_2 ; "taskkill.exe /f /im sqlwriter.exe"
.text:10005906 call    sub_10001080
.text:1000590B push    0               ; lpExitCode
.text:1000590D push    0               ; dwMilliseconds
.text:1000590F push    offset aTaskkillExeFIm_3 ; "taskkill.exe /f /im mysqld.exe"
.text:10005914 call    sub_10001080
.text:10005919 add     esp, 3Ch
```

Figure 4-4

5 Discussion

5.1 General Discussion

The outcome of the analysis conducted did not meet the aim as well as was initially expected.

While it was found DLL module proved to include a substantial amount of information that was useful to understand the activities performed by the sample, far too much time was spent attempting to retrieve the means to decrypt it.

This led later to many sacrifices regarding the analysis performed afterward and left many unexplored areas, especially regarding the performed network activities both due to time constraints and unexpected behavior of the tools used.

In general, a substantial number of the steps performed by the sample were recovered. Also, the recovered steps cover mostly the main purpose of ransomware which is the encryption of user files, generation of the keys used for that purpose, information about the file format used for storing both the encrypted file and the key used for encryption and decryption, and some other related activity performed on the system to better support that goal.

The sample itself showcases how cryptography in ransomware can lead to serious damage to the owner of digital assets.

Generating a different AES key for each file and encrypting those keys with an RSA encryption, this limits in high amounts any recovery of files encrypted by the sample.

This sample in the amount that was possible to analyze, did not contain any autonomous propagation to other machines through the network. This means first that this is a variant of the original WannaCry ransomware. This also makes the sample in a corporate environment, if it is regularly performed a backup of the data, much less dangerous than the original version as the data will not be lost and can be restored. Also, the machine can be restored easily, especially in a virtual environment where constant snapshots of the machine are made.

However, any drive connected to an infected host which is not a DVD drive, will be at risk. As soon as detected by the ransomware-spawned threads, encryption on the drive will start and data will be lost.

This can mean that in a corporate environment where shared storage is used across several machines, such a medium will be at risk if not properly protected.

The impact on the user can be equally disruptive for the end consumer regardless of its lack of ability to propagate on the network.

In corporate environments usually, backups of data and backup systems are in place to have continuous and uninterrupted service and protect customer data. While propagation in a corporate environment can

change drastically the amount of threat the sample represents as even those systems could be compromised, meaning a real loss of information, the end user differs in the way it is affected by this type of software.

An end user does not always rely on backup solutions meaning that infection on his device would lead to irreversible damage to his personal information. Even if the malware is not capable of spreading from his machine to others, users in such a category would not have any ability to restore the data which is likely to be centralized on the compromised machine.

This makes this sample especially dangerous for the end consumer, but much less dangerous in general for corporate entities with strong backup solutions and systems.

5.2 Countermeasures

The sample has many signatures that can be easily identified by any AV software. All encrypted files use the WNCRY extension which together with the “WANNACRY!” string is found in the executable modules or embedded files such as t.wnry.

Because of the inability of the sample to propagate on the network having proper backup solutions is already enough strong mitigation in many environments.

However, proper protection of shared storage should be applied as the sample will scan for any drive connected and encrypt the ones which are not DVD drives and have write permissions enabled.

5.3 Future Work

As described during analysis far too many aspects were not possible to be thoroughly analyzed as initially desired.

Most of the uncovered aspects were left behind due to time constraints present especially due to the high amount of time spent in the decryption of the DLL module.

Some of the aspects which were left uncovered are:

- Better dynamic and static analysis of the base executable and DLL module
- Analysis of the taskse.exe and taskdl.exe executables
- Analysis of the decryption tool
- Recovery of the RSA decryption key used for decryption of the AES key used to recover the DLL module
- In-depth network analysis
- Better analysis of artifacts left by the execution
- Monitor for any overlooked system configuration change

These and other aspects that during analysis were not possible to perform might be possible to be performed in the future to better understand activity on the victim machine during the execution of the sample.

REFERENCES

[1] Sean, K. M. [Online]. Available from: https://www.microsoft.com/en-us/security/blog/2017/05/12/wannacrypt-ransomware-worm-targets-out-of-date-systems/?source=mmpc [Accessed 21 April 2023].

[2]Micheal, K. [Online]. Available from: https://www.pcworld.com/article/406793/paying-the-wannacry-ransom-will-probably-get-you-nothing-heres-why.html [Accessed 21 April 2023].

[3] Matt, G. NTInfo. [Online]. Available from: https://horsicq.github.io/ [Accessed 21 April 2023].

APPENDICES

Note that Appendices should be referenced in the main body of the text.

APPENDIX A

5.3.1 XIA Archive Content Identification

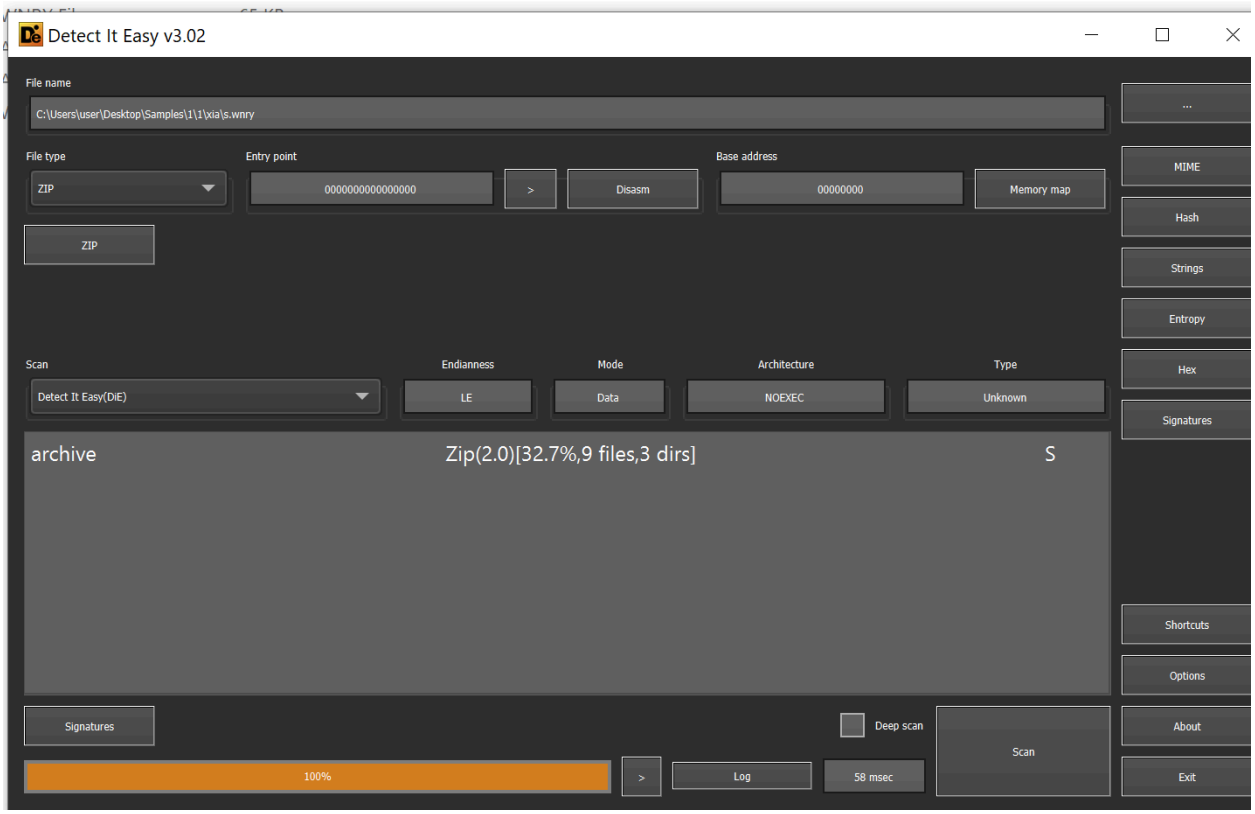


Figure A-1 Tor Zip Identification

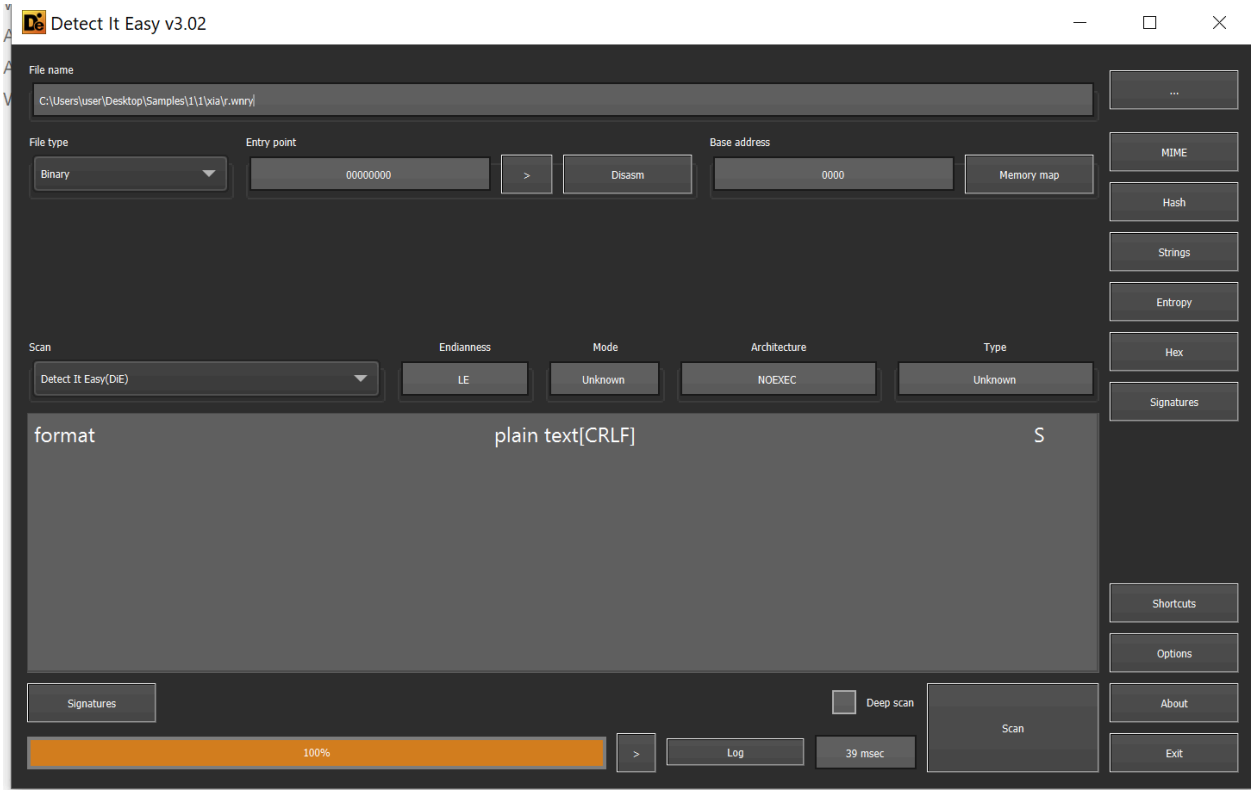


Figure A-2 Text File for instructions Identification

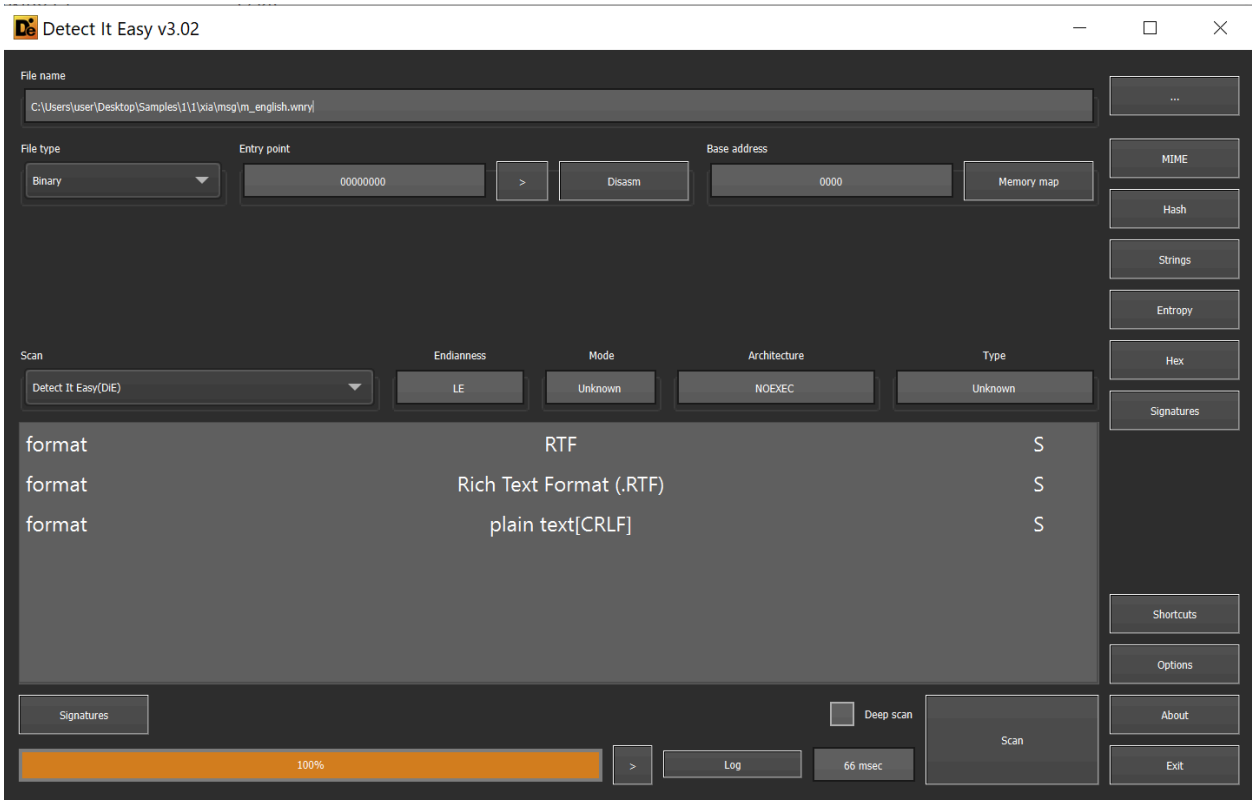


Figure A-3 Documents in MSG folder identification

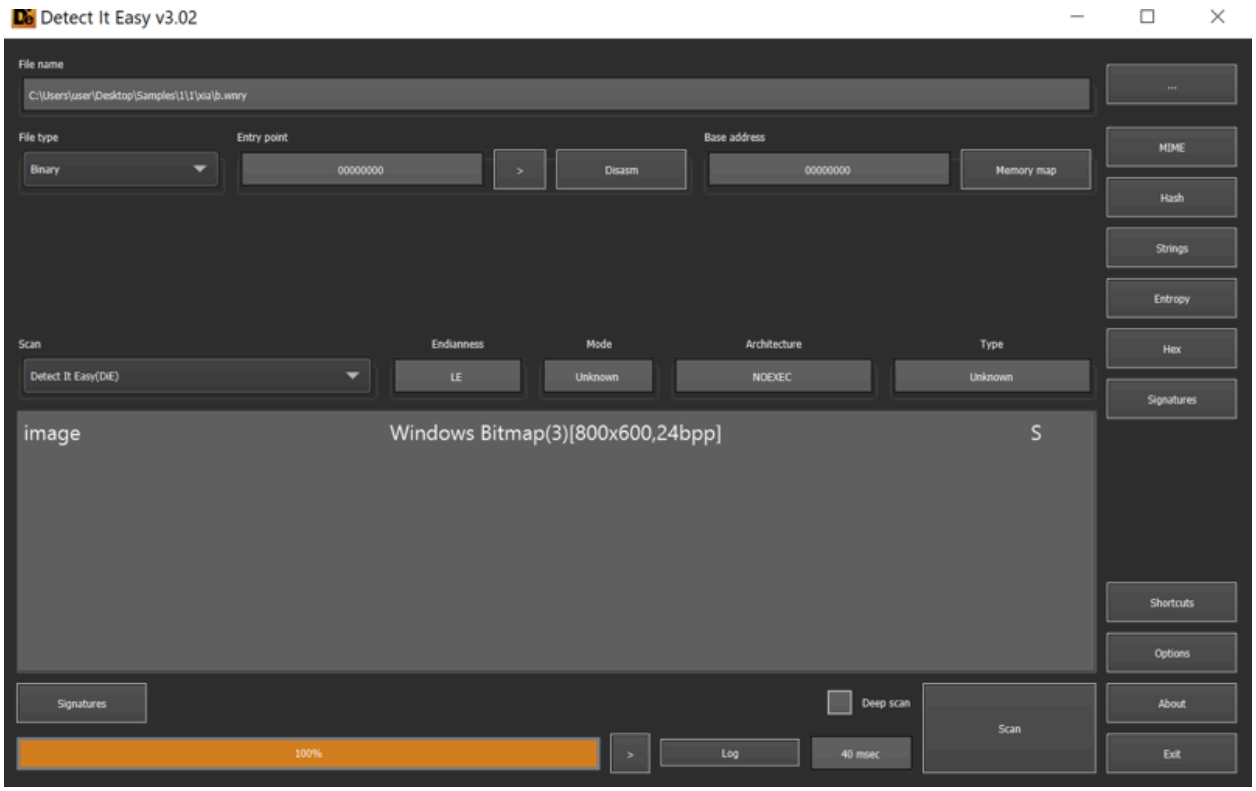


Figure A-4 Wallpaper Identification

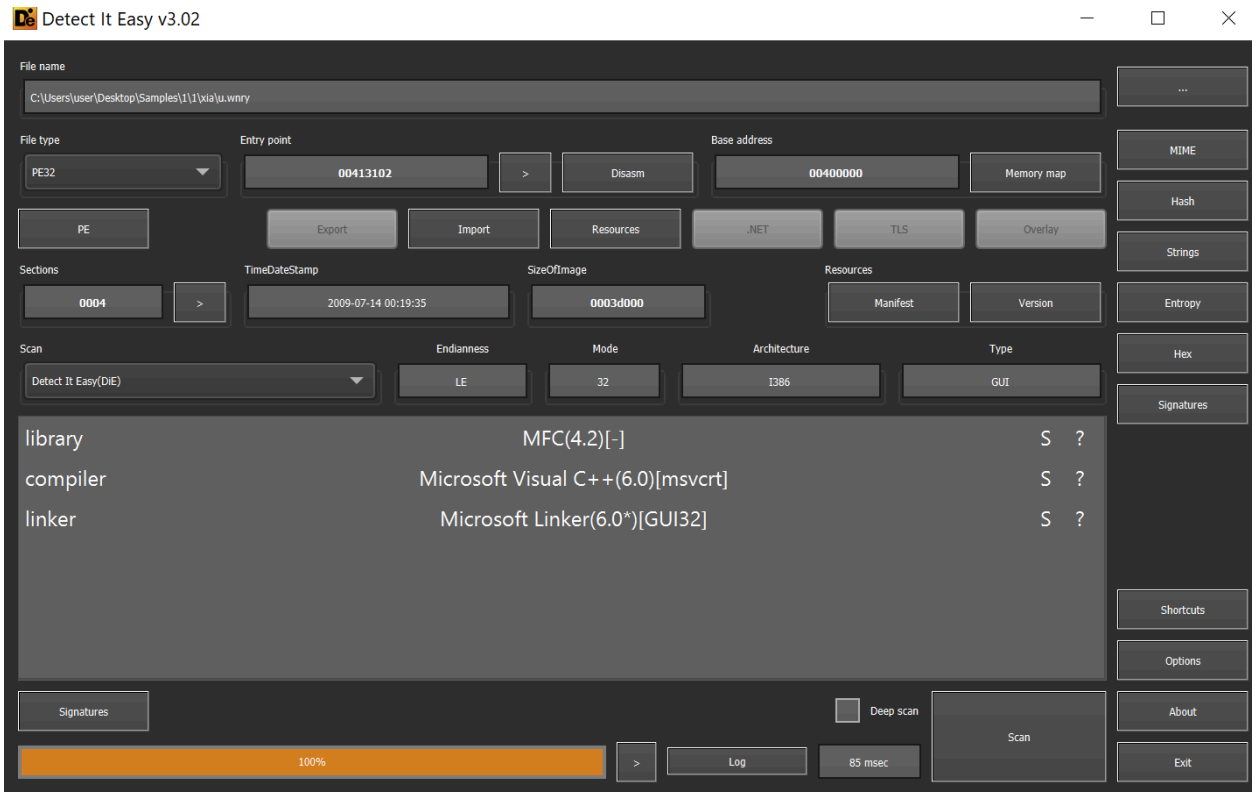


Figure A-5 Decryption Tool Identification

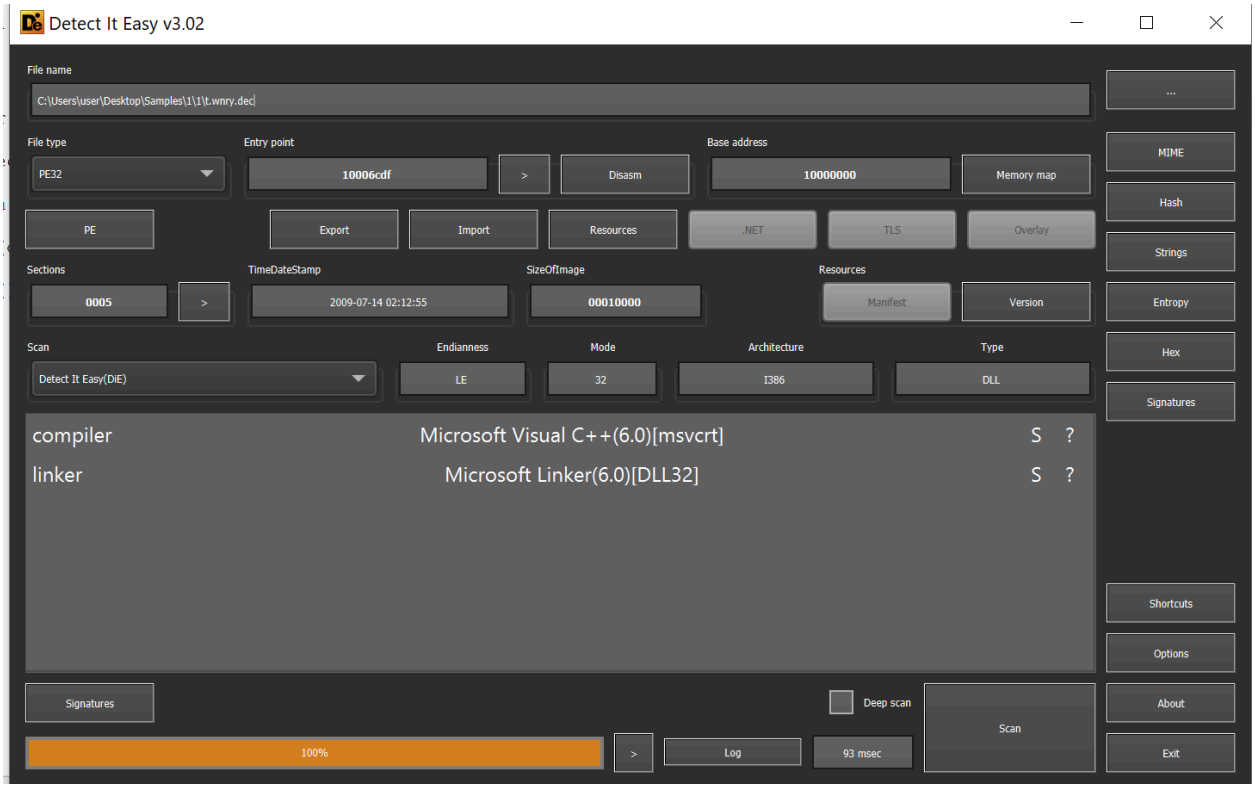


Figure A-6 DLL Identification using the DIA tool

5.3.2 General Screenshots

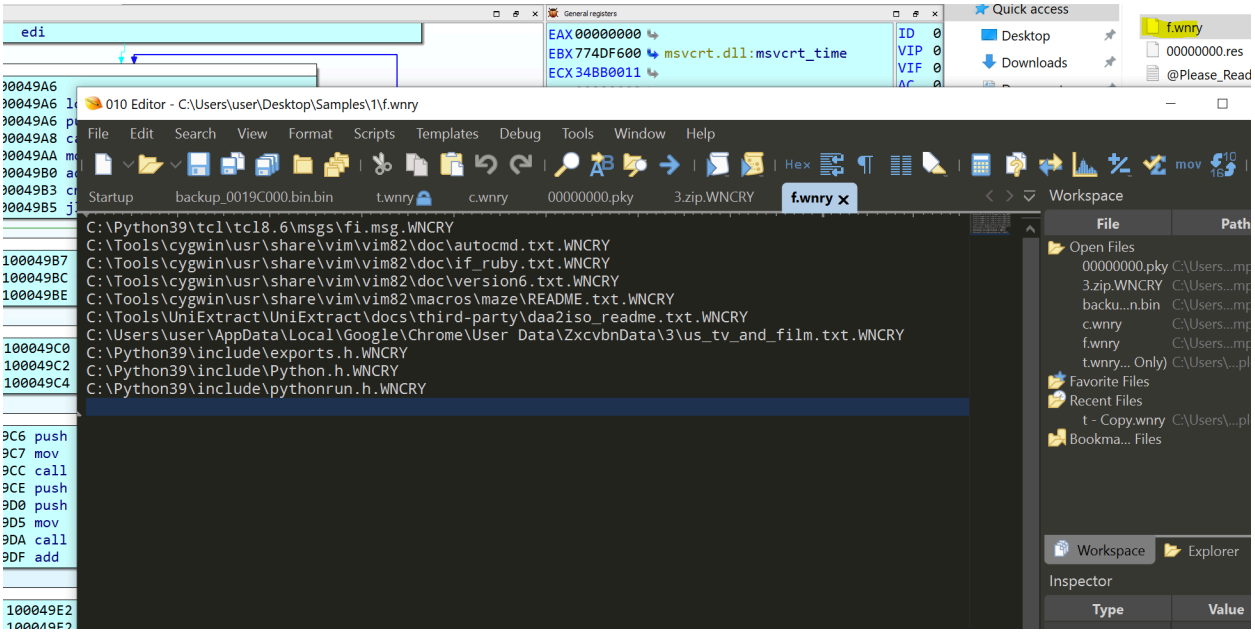


Figure A-7 List generation of files to be decrypted for free



Figure A-8 – VBS Script for creating a shortcut on the Desktop to the decryption tool of the ransomware

VBS script which generates a shortcut for the decryption tool on the desktop.

```
.text:1000481E
.text:1000481E loc_1000481E:
.text:1000481E mov     al, byte_1000DD98
.text:10004823 mov     ecx, 18h
.text:10004828 mov     [esp+4A0h+var_464], al
.text:1000482C xor     eax, eax
.text:1000482E lea     edi, [esp+4A0h+var_463]
.text:10004832 rep stosd
.text:10004834 stosw
.text:10004836 lea     ecx, [esp+4A0h+var_464]
.text:1000483A push    ecx
.text:1000483B stosb
.text:1000483C call    sub_100014A0
.text:10004841 mov     edx, [esp+4A4h+arg_0]
.text:10004848 add     esp, 4
.text:1000484B lea     eax, [esp+4A0h+var_464]
.text:1000484F lea     ecx, [esp+4A0h+var_498]
.text:10004853 push    edx
.text:10004854 push    eax
.text:10004855 push    ecx
.text:10004856 lea     edx, [esp+4ACh+Buffer]
.text:1000485D push    offset aCmdExeCRegAddS ; "cmd.exe /c reg add %s /v \"%s\" /t REG_"...
.text:10004862 push    edx             ; Buffer
.text:10004863 call    ds:sprintf
.text:10004869 push    0               ; lpExitCode
.text:1000486B lea     eax, [esp+4B8h+Buffer]
.text:10004872 push    2710h           ; dwMilliseconds
.text:10004877 push    eax             ; lpCommandLine
.text:10004878 call    sub_10001080
.text:1000487D add     esp, 20h
.text:10004880 pop     edi
.text:10004881 pop     esi
```

Figure A-9 – cmd command used to manipulate the Windows registry

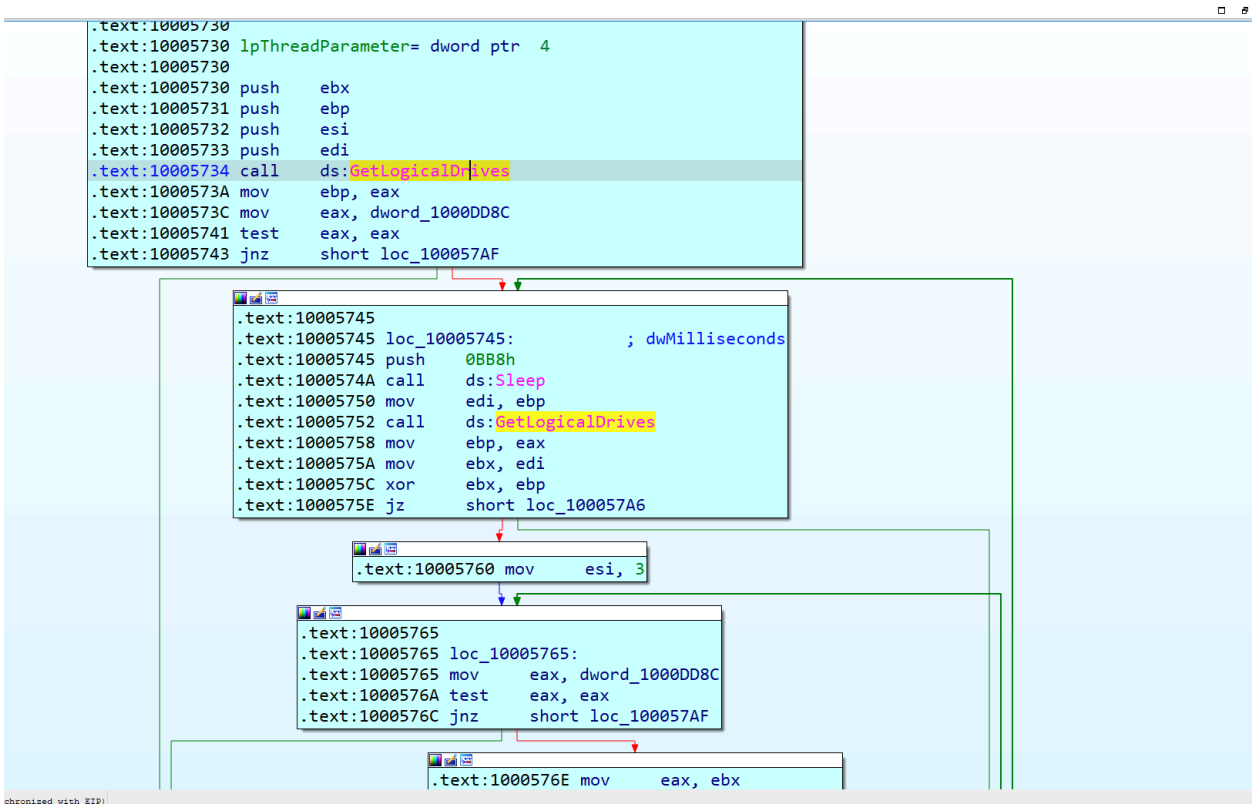


Figure A-10 – Code used by a thread to scan for logical drives connected to the machine each 3 seconds

APPENDIX B

```
import os
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

key = bytes.fromhex("BEE19B98D2E5B12211CE211EECB13DE6")

iv = 16 * b'\x00'

f = open ("t.wnry.enc", "rb").read()

decryptor = Cipher(algorithms.AES128(key),modes.CBC(iv)).decryptor()

data = decryptor.update(f) + decryptor.finalize()

f2 = open("t.wnry.dec", "wb")

f2.write(data)

f2.close()
```

Figure B-1 Python Script used for DLL decryption